

Choose the correct answer in each of the following:

- 1) Which component links different object code files into a single executable?
 - a) Assembler
 - b) Linker
 - c) Preprocessor
 - d) Parser

- 2) What is the role of semantic analysis in compilation?
 - a) Token generation
 - b) Linking object code
 - c) Generating executable code
 - d) Type checking and detecting semantic errors

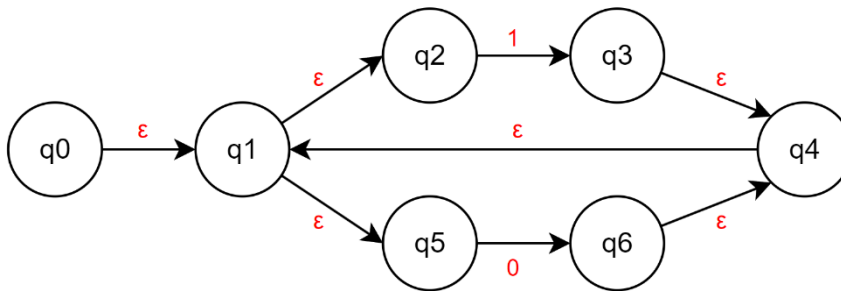
- 3) Which of the following is TRUE about the final phase of the compiler?
 - a) Optimizes the intermediate code.
 - b) Generates object code from optimized intermediate code.
 - c) Converts object code back to source code.
 - d) Generates three-address code.

- 4) What kind of error is detected by the lexical analyzer?
 - a) Undefined variables
 - b) Incorrect data types
 - c) Illegal characters
 - d) Stack overflow

- 5) Which compiler phase is responsible for reporting most syntax errors?
 - a) Semantic Analyzer
 - b) Code Generator
 - c) Lexical Analyzer
 - d) Syntax Analyzer

Q2

a. Convert the following NFA into its equivalent DFA. (Fill the table)



Details:

$\epsilon\text{-closure}(q0) = \{q0, q1, q2, q5\} = A$

$\epsilon\text{-closure}(\text{move}(A,0)) = \epsilon\text{-closure}(q6) = \{q1, q2, q4, q5, q6\} = B$

$\epsilon\text{-closure}(\text{move}(A,1)) = \epsilon\text{-closure}(q3) = \{q1, q2, q3, q4, q5\} = C$

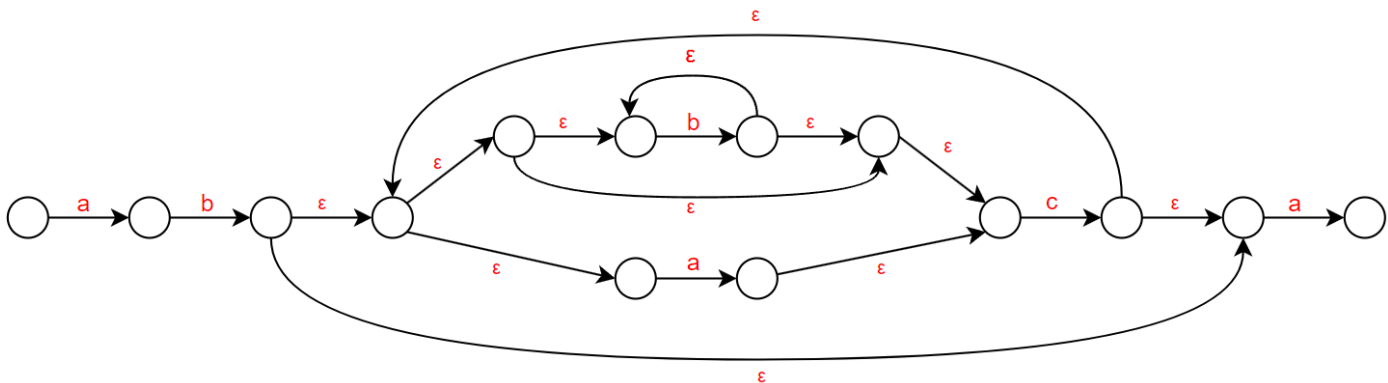
$\epsilon\text{-closure}(\text{move}(B,0)) = \epsilon\text{-closure}(q6) = B$

$\epsilon\text{-closure}(\text{move}(B,1)) = \epsilon\text{-closure}(q3) = C$

NFA State	DFA State	0	1
$\{q0, q1, q2, q5\}$	A	B	C
$\{q1, q2, q4, q5, q6\}$	B	B	C

b. Using **McNaughton-Yamada-Thompson** algorithm convert the regular expression to NFA:

$ab(a|b^*c)^*a$



Q3

Eliminate left recursion (**only one very easy grammar**)

Q4

- a. Construct the canonical LR(0) collection for the following grammar using CLOSURE and GOTO. Stop after completing the states which are coming from (I_0)

$S' \longrightarrow S$
 $S \longrightarrow AaAb$
 $S \longrightarrow BbBa$
 $A \longrightarrow b$
 $B \longrightarrow a$

$I_0:$ $S' \longrightarrow .S$
 $S \longrightarrow .AaAb$
 $S \longrightarrow .BbBa$
 $A \longrightarrow .b$
 $B \longrightarrow .a$

$\text{GoTo}(I_0, A)$

$I_2:$ $S \longrightarrow A.aAb$

$\text{GoTo}(I_0, a)$

$I_5:$ $B \longrightarrow a.$

$\text{GoTo}(I_0, B)$

$I_3:$ $S \longrightarrow B.bBa$

$\text{GoTo}(I_0, S)$

$I_1:$ $S' \longrightarrow S.$

$\text{GoTo}(I_0, b)$

$I_4:$ $A \longrightarrow b.$

- b. What are the final states

I_1 I_4 I_5

Q5

- $E \rightarrow T E'$
- $E' \rightarrow + T E' \mid \varepsilon$
- $T \rightarrow F T'$
- $T' \rightarrow * F T' \mid \varepsilon$
- $F \rightarrow (E) \mid id$

Given the above grammar Show the sequence of procedure calls (and returns) performed by a recursive descent parser for the following:

E: (Case1: It has **only one** production → **write the production as it is. No if-else if or switch**)

```
E(){  
    T(); E'();  
}
```

T': (Case2: It has **more than one** production **with ε** → **if-else if or switch *without error report***)

```
T'(){  
    If(lookahead == '*')  
        match( '*' ); F(); T'();  
}
```

F: (Case3: It has **more than one** production **without ε** → **if-else if or switch *with error report***)

```
F(){  
    switch( lookahead ) {  
        case '(': match('('); E(); match(')'); break;  
        case 'id': match('id'); break;  
        default: report( 'syntax error' );  
    }  
}
```

Q6

$$E \rightarrow E+T \mid T$$

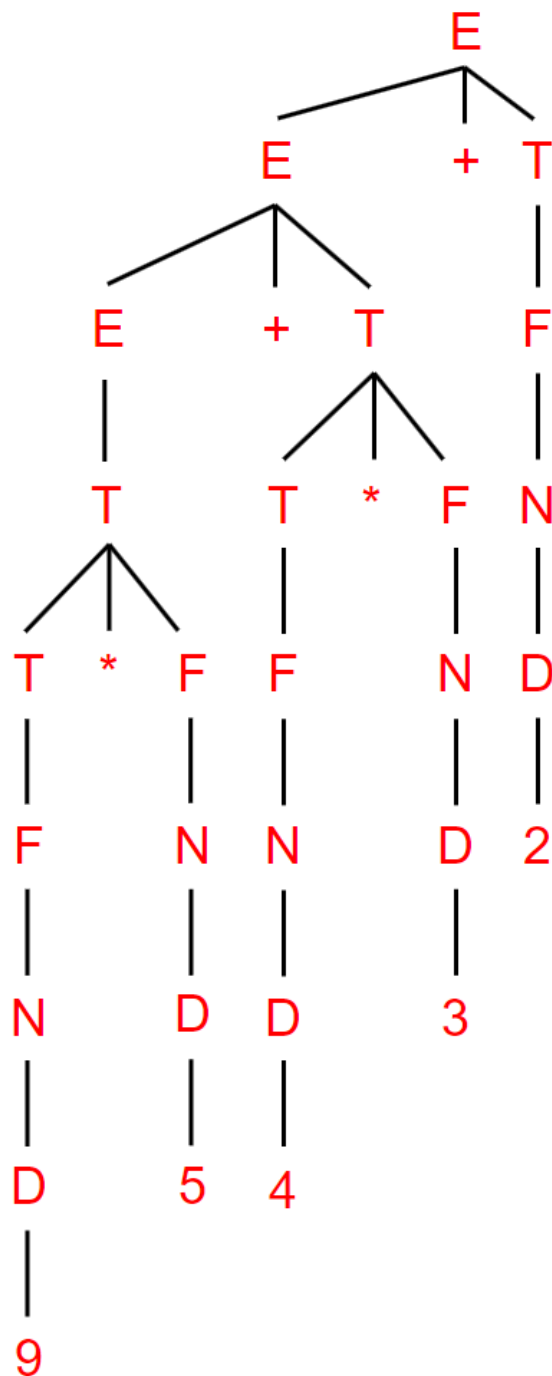
$$T \rightarrow T * F \mid F$$

$$F \rightarrow N \mid E$$

$$N \rightarrow D$$

$$D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

(a) Draw pars tree for: $9 * 5 + 4 * 3 + 2$



(b) Find the left derivation for: $8 * 5 + 9$

$$E \Rightarrow E + T$$

$$E \Rightarrow T + T$$

$$T \Rightarrow T * F + T$$

$$T \Rightarrow F * F + T$$

$$F \Rightarrow N * F + T$$

$$N \Rightarrow D * F + T$$

$$D \Rightarrow 8 * F + T$$

$$F \Rightarrow 8 * N + T$$

$$N \Rightarrow 8 * D + T$$

$$D \Rightarrow 8 * 5 + T$$

$$T \Rightarrow 8 * 5 + F$$

$$F \Rightarrow 8 * 5 + N$$

$$N \Rightarrow 8 * 5 + D$$

$$D \Rightarrow 8 * 5 + 9$$

(c) is the grammar ambiguous for $(*, +)$? explain your answer.

For only the first two productions (with $+$ and $*$), the grammar is **not ambiguous**; it already encodes precedence and left associativity.

Q7

(a)

he gave us different grammar

Q. No. 2: Find FIRST and FOLLOW sets for the following grammar and then construct top-down non-recursive Predictive Parse Table for it.

SO: 1 CLO: 10 Total Points: 3 Obtained Points: 2.5

Grammar:

$$\begin{aligned} S &\rightarrow aBA \mid b \\ A &\rightarrow SaB \mid Bb \mid \epsilon \\ B &\rightarrow cB \mid Sd \mid \epsilon \end{aligned}$$

Solution:

First(S) = {a, b} ✓
 First(A) = {a, b, c, null} ✓
 First(B) = {c, a, b, null} ✓

Follow(S) = {\$, a, d} ✓
 Follow(A) = {\$, a, d} ✓
 Follow(B) = {\$, a, d, b} ✓

Parse Table:

Non-Terminals	a	b	c	d	S
S	S → aBA Sync	S → b ✓		Sync	Sync
A	S → aBA A → ε	S → b	B → cB	A → ε ✓	A → ε
B	S → aBA Sync	S → b	B → cB	B → ε ✓	B → ε ✓

(b)

Only difference here is he gave us S only once ($S \rightarrow id \mid V = E$) and same for E ($E \rightarrow V \mid n$)

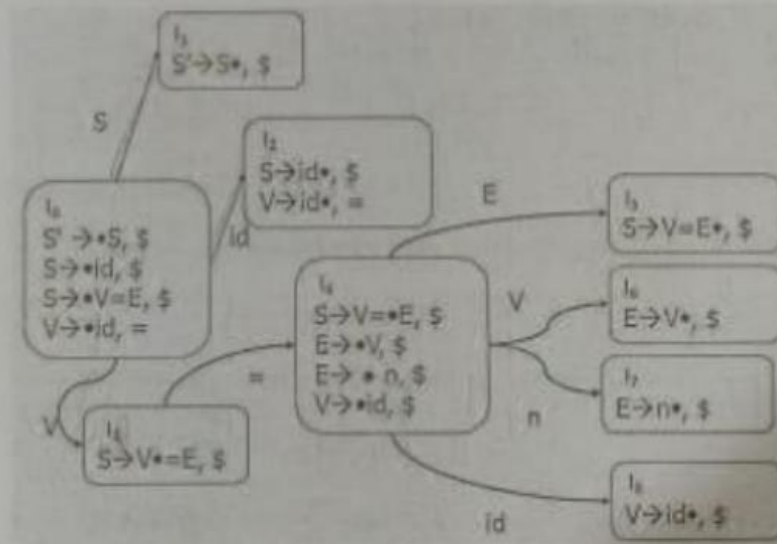
(b) Consider the following augmented grammar:

- $S' \rightarrow S$
- 1) $S \rightarrow id$
- 2) $S \rightarrow V = E$
- 3) $V \rightarrow id$
- 4) $E \rightarrow V$
- 5) $E \rightarrow n$

(10 points)

10

The Canonical LR(1) items for this grammar are given in the following transition diagram.



Construct LR(1) parse table for the above grammar using this transition diagram.

Solution:

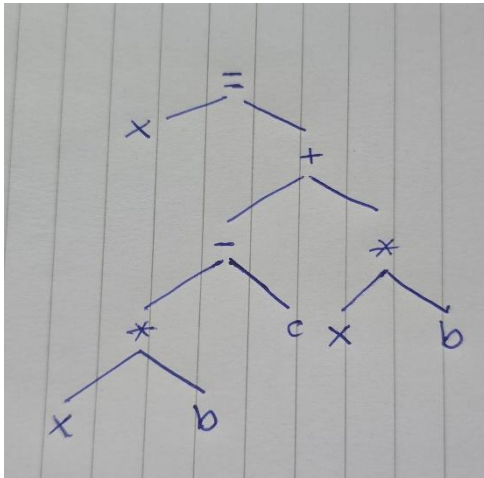
STATE	ACTIONS				GOTO		
	Id	=	n	S	S	V	E
0	S2	R1			1	3	
1				Accept			
2		R3		R1			

Q8

Consider the following assignment statement.

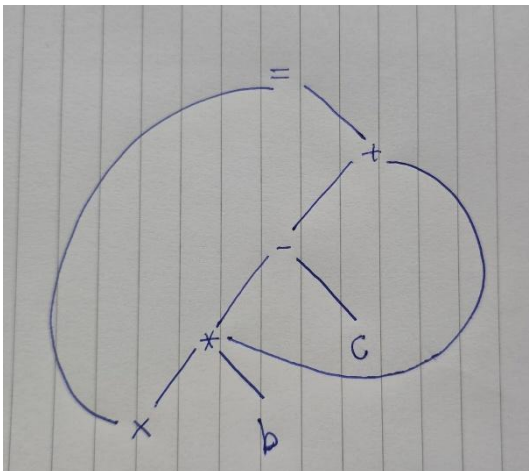
$$x = x * b - c + x * b$$

- a. Construct abstract syntax tree (AST) for this statement.



Number of nodes in the AST: 11

- b. Draw directed acyclic graph (DAG) for the above assignment statement.



Number of nodes in the DAG: 7

c. Represent the above assignment statement in three address intermediate code

$$b = (b + c) * c - d$$

$$t1 = b + c$$

$$t2 = t1 * c$$

$$t3 = t2 - d$$

$$b = t3$$

Show the quadruple representation of the above three

op	arg1	arg2	result
+	b	c	t1
*	t1	c	t2
-	t2	d	t3
=	t3		b